

Secure Multi-Tenant Activity Feed - Submission Report

- **Candidate:** Pritam Raha
- **Stack:** Next.js 16, TypeScript, Supabase Auth, PostgreSQL, Row Level Security
- **Repository:** https://github.com/pritam16raha/01_Know_your_organization
- **Live application:** <https://01-know-your-organization.vercel.app/login>

1. Summary

This submission delivers the requested multi-tenant activity-feed vertical slice. A reviewer can select either of two seeded demo identities, sign in through real Supabase Auth, view only that organization's accounts, read account notes newest first, and add a note. The UI includes loading, empty, success, validation, authorization, and server-error feedback.

Tenant isolation is enforced in PostgreSQL rather than being inferred from the dropdown or trusted browser data. Note creation is also duplicate-safe: retrying the same intended operation returns the original activity instead of inserting another row.

2. Architecture

TEXT

```
Browser
-> Next.js route handlers (cookie session + Zod validation)
  -> Supabase Data API / PostgreSQL RPCs (authenticated user's JWT)
    -> RLS membership policies + tenant-consistent constraints
      -> organizations / memberships / accounts / activity_entries
```

Next.js provides the frontend and the required backend-for-frontend operations. `proxy.ts` refreshes the Supabase session and performs only optimistic navigation checks; each route handler authenticates again before accessing data. The route handlers expose workspace discovery, newest-first activity reads, and validated note creation.

The data model is intentionally small:

- `organizations` is the tenant root.
- `memberships` associates each real `auth.users` identity with one organization for this assessment.
- `accounts` belongs to an organization.
- `activity_entries` stores the account, derived organization, derived author, body, timestamp, and idempotency key.

Northstar Labs owns Acme Corporation and Globex Retail. Rival Systems owns Rival Confidential Account. Pritam Raha belongs to Northstar; Alex Rival belongs to Rival Systems. The account dropdown therefore shows two accounts to Pritam and one to Alex.

3. Security approach

- RLS is enabled on every application table. Policies resolve membership from `auth.uid()`.
- The browser never supplies or selects an `organization_id`. It sends only the chosen account ID, note body, and idempotency key.
- PostgreSQL derives `organization_id` and `created_by` from the authenticated user and visible account. Insert grants prevent callers from setting protected columns.
- Composite foreign keys prevent an activity from claiming a tenant different from its account or author membership.
- Missing and cross-tenant account IDs intentionally return the same message, limiting account enumeration.
- Normal requests use the authenticated user's JWT and publishable Supabase key. A service-role key is obtained transiently only by the local seed script and is never stored in the application or repository.
- Note input is trimmed and validated at both the TypeScript and PostgreSQL boundaries, with a maximum length of 2,000 characters.

- A unique `(organization_id, idempotency_key)` constraint is the concurrency-safe duplicate boundary. Reusing a key for a different request is rejected.
- Quick-access credentials are throwaway evaluator identities. Hosted values come from server-side environment variables, with the ignored local credential file used only for local development. A partial hosted configuration fails closed.

4. Tests and evidence

Automated verification exercises the database, HTTP, client, and browser layers:

BASH

```
cd Frontend
npm run typecheck
npm run lint
npm run build
npm run verify:integration
npm run verify:client
```

The HTTP and real-browser checks run against a started production server. `README.md` documents the required port and `TEST_BASE_URL` setup for `npm run verify:http` and `npm run verify:browser:lan`.

Observed results:

Scenario	Expected and observed result
Unauthenticated workspace	HTTP 401
Authorized activity read	HTTP 200 , newest first
First note creation	HTTP 201 , <code>wasDuplicate: false</code>
Identical retry	HTTP 200 , same activity ID, <code>wasDuplicate: true</code>
Northstar reads Rival account	HTTP 403 ; raw PostgreSQL 42501
Northstar writes Rival account	HTTP 403 ; raw PostgreSQL 42501 ; no row created
Database lint	No findings
TypeScript, ESLint, production build	Passed
Real Edge browser on insecure LAN origin	Login, Globex create, visible success, cleared form, and one rendered note passed
Hosted demo configuration	Complete environment renders both tabs; partial environment fails closed

The automated checks are reproducible in `Frontend/scripts/`. Appendix A below provides evaluator-friendly manual checks without adding a cross-tenant option to the UI.

5. Trade-offs, incomplete work, and another two hours

The mandatory vertical slice is complete. The assessment models one active organization per user, so `memberships.user_id` is unique. A larger product that allows one identity in multiple organizations would require an explicit organization selector backed by signed or server-maintained active-tenant context.

Intentionally incomplete work includes pagination, tenant switching, role administration, password recovery, generated database types, CI deployment checks, request tracing, broader browser coverage, and production telemetry. Screenshots and a short walkthrough video are being supplied separately as submission evidence rather than stored in the application repository.

With another two hours, I would prioritize:

1. Playwright coverage for both tenants, validation errors, session expiry, and mobile layouts.
2. Generated Supabase TypeScript types plus CI gates for lint, type checking, build, and integration tests.
3. Pagination and request correlation IDs.

4. Structured monitoring of `401`, `403`, and PostgreSQL `42501` ratios by route and release, excluding credentials, cookies, tokens, note bodies, and idempotency keys.

The required implementation and real-browser verification were completed from **12:40:21 to 13:38:26 IST (58 minutes)**. A separately recorded post-timebox correction at **15:14 IST** added hosted demo-environment support after deployment exposed the local-file assumption. The complete AI collaboration record is in [@@HTML_TOKEN_0@@](#).

6. Development process and AI collaboration

Before writing application code, I used ChatGPT to review the assignment, clarify the architecture and tenant-security requirements, identify uncertainties, and define the implementation and verification plan. I had also prepared a standalone HTML UI prototype before starting the application implementation. Refactoring that prototype into Next.js and TypeScript reduced UI delivery time and allowed more of the timebox to be spent on Supabase Auth, PostgreSQL RLS, idempotency, error handling, and tests.

- **Shared planning and implementation conversation:** <https://chatgpt.com/share/6a771f5f-3f8c-83ee-9a30-e969e5791a6f>
- **Structured AI audit trail:** [@@HTML_TOKEN_0@@](#)

The repository history and `AI_USAGE.md` record the important prompts, accepted/changed/rejected suggestions, verification steps, and corrections made during implementation.

7. Suggested accompanying evidence

Attach these separately or embed selected images before exporting this report to PDF:

1. Login page showing both Quick Demo Access tabs.
 2. Pritam's dropdown showing Acme Corporation and Globex Retail.
 3. Alex's dropdown showing only Rival Confidential Account.
 4. Successful note creation with visible success feedback.
 5. DevTools output for cross-tenant HTTP `403` and the identical-retry comparison.
 6. Supabase SQL Editor output showing raw PostgreSQL `42501`.
 7. A short video showing both tenant views, a successful note, a denied tampered request, and an identical retry.
-

Appendix A - Manual security and idempotency verification

Unauthorized accounts are intentionally absent from the dropdown. This is expected UI behavior, not the security boundary. Use the browser Console to simulate a tampered request and confirm that the API and database still deny access.

A.1 Cross-tenant read

Sign in as **Pritam Raha**, open **DevTools** -> **Console**, and run:

JAVASCRIPT

```
const rivalAccountId = "30000000-0000-4000-8000-000000000003";

const response = await fetch(
  `/api/accounts/${rivalAccountId}/activities`,
  { credentials: "same-origin" }
);

console.log("Status:", response.status);
console.log("Response:", await response.json());
```

Expected:

TEXT

```
Status: 403
Response: {
  error: "Account not found or is not accessible."
}
```

Pritam belongs to Northstar Labs, while this account belongs to Rival Systems.

A.2 Cross-tenant create

While still signed in as **Pritam Raha**, run:

JAVASCRIPT

```
const rivalAccountId = "30000000-0000-4000-8000-000000000003";

const response = await fetch(
  `/api/accounts/${rivalAccountId}/activities`,
  {
    method: "POST",
    credentials: "same-origin",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      body: "This cross-tenant note must be rejected.",
      idempotencyKey: "f0000000-0000-4000-8000-000000000001"
    })
  }
);

console.log("Status:", response.status);
console.log("Response:", await response.json());
```

Expected:

TEXT

```
Status: 403
Response: {
  error: "Account not found or is not accessible."
}
```

No note will be created.

A.3 Identical retry

While signed in as **Pritam Raha**, use Globex Retail because it belongs to Northstar Labs:

JAVASCRIPT

```
const globexAccountId = "30000000-0000-4000-8000-000000000002";

const payload = {
  body: "Manual idempotency test.",
  idempotencyKey: "7f30e0c8-f741-4dde-8cce-00b5f1549a01"
};

async function createNote() {
  const response = await fetch(
    `/api/accounts/${globexAccountId}/activities`,
    {
      method: "POST",
      credentials: "same-origin",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(payload)
    }
  );

  return {
    status: response.status,
    data: await response.json()
  };
}

const first = await createNote();
const retry = await createNote();

console.log("First request:", first);
console.log("Retry:", retry);
console.log(
  "Same activity ID:",
  first.data.activity.id === retry.data.activity.id
);
```

Expected first request:

TEXT

```
status: 201
wasDuplicate: false
```

Expected retry:

TEXT

```
status: 200
wasDuplicate: true
Same activity ID: true
```

The two `activity.id` values must be identical. If the complete test is repeated later, change the idempotency key; otherwise the first request will also be recognized as a retry.

A.4 Raw PostgreSQL 42501

The Next.js API intentionally maps database `42501` to HTTP `403` so internal database details are not exposed to the frontend.

Open the **Supabase SQL Editor** and run the following cross-tenant read test:

SQL

```
begin;

select set_config(
  'request.jwt.claim.sub',
  (
    select id::text
    from auth.users
    where email = 'pritam@northstar.test'
  ),
  true
);

set local role authenticated;

select *
from public.get_account_activity(
  '30000000-0000-4000-8000-000000000003'
);

rollback;
```

Expected:

TEXT

```
ERROR: 42501: Account not found or is not accessible.
```

Then run the cross-tenant create test:

SQL

```
begin;

select set_config(
  'request.jwt.claim.sub',
  (
    select id::text
    from auth.users
    where email = 'pritam@northstar.test'
  ),
  true
);

set local role authenticated;

select *
from public.create_account_note(
  '30000000-0000-4000-8000-000000000003',
  'This must be denied.',
  'f0000000-0000-4000-8000-000000000002'
);

rollback;
```

Expected:

TEXT

```
ERROR: 42501: Account not found or is not accessible.
```

If the SQL Editor stops at the expected error before executing `rollback`, run `rollback;` separately before the next test.

The two statuses represent the same authorization denial at different layers:

TEXT

```
PostgreSQL/RLS 42501
|
v
Next.js error mapping
|
v
HTTP 403
```